



A novel freemium code SAND (v1.0) for generation of randomly packed beds

Nikita Shadymov ^a, Viacheslav Papkov ^{c, b}, Dmitry Pashchenko ^{b, c, d, *}

^a Samara State Technical University, Molodogvardeiskaya 244 St, Samara, 443100, Russia

^b Department of Mechanical Engineering (Robotics), Guangdong Technion-Israel Institute of Technology, 241 Daxue Road, Shantou, Guangdong, 515063, China

^c Faculty of Mechanical Engineering, Technion - Israel Institute of Technology, Technion City, Haifa, 3200003, Israel

^d Guangdong Provincial Key Laboratory of Materials and Technologies for Energy Conversion, Guangdong Technion—Israel Institute of Technology, Guangdong, 515063, China



ARTICLE INFO

Article history:

Received 28 August 2024

Received in revised form

10 September 2024

Accepted 10 September 2024

Available online 5 October 2024

Keywords:

Packed bed

Discrete element method

Python-based code

CAD software

ABSTRACT

The main current approaches for generation of the packed bed models are based on rigid body dynamics (RBD) and Newton's laws (discrete element methods - DEM). This paper deals with the development and analysis of a novel code based on analytical geometry approach for the packed bed generation. The architecture and main algorithms of the novel code are described and clarified. The parameters of the packed bed generated via the novel code are compared with experimental data and packed beds generated via Blender (RBD), Yade (DEM). The novel code demonstrates many advantages, such as good correlation with experimental data, no overlaps between pellets in the packed bed, and a low computational time for packed bed generation. The packed bed model can be directly exported in .step format. Other advantages are also demonstrated and clarified. The novel code is attached to this paper and can be freely used by engineers and scientists.

© 2024 Chinese Society of Particuology and Institute of Process Engineering, Chinese Academy of Sciences. Published by Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

1. Introduction

A packed bed refers to a container or column filled with solid particles or structured packing material, widely used in chemical engineering processes such as absorption, distillation, and catalytic reactions (Krishnamurthy & Taylor, 1985; Sánchez et al., 2016; Xiuping et al., 2008). The solid particles provide a large surface area for contact between the fluid phase (liquid or gas) and the solid phase, facilitating mass transfer, heat transfer, and chemical reactions (Gaber et al., 2018; Habib et al., 2021; Mokheimer et al., 2024). Studying packed beds is essential for optimizing process efficiency, understanding reaction kinetics, controlling pollutants, designing scalable systems, and minimizing environmental impact in various chemical engineering applications (Jafari et al., 2008; Nemec & Levec, 2005; Noorman et al., 2007).

* Corresponding author. Department of Mechanical Engineering (Robotics), Guangdong Technion-Israel Institute of Technology, 241 Daxue Road, Shantou, Guangdong, 515063, China.

E-mail address: dmitry.pashchenko@gtit.edu.cn (D. Pashchenko).

The packed beds for computational fluid dynamics (CFD) have two main forms: a porous medium approach (Diersch & Kolditz, 2002; Wang et al., 2010) and a particle-resolved approach (Dixon & Partopour, 2020; Jurtz et al., 2019; Tong et al., 2020). In the first approach, the packed bed is represented via a pseudo-homogeneous model. In the particle-resolved approach, a packed bed represents as many individual pellets. These packed beds consist of randomly filled hundreds and thousands of particles. Therefore, generation of 3D model of packed bed is crucial task (Afandizadeh & Foumeny, 2001; Ohshima et al., 1997; Tong et al., 2020).

Many algorithms for creating packed beds via a particle-resolved approach have been proposed, such as the rigid body dynamics (RBD) (Costello et al., 2007; Fleischlen & Wehinger, 2019), an algorithm for creating a single-layer packed bed (Karpilov & Pashchenko, 2023; Moghaddam et al., 2018), the discrete element method (DEM) (Kloss et al., 2012; Munjiza, 2004; Zhong et al., 2016), etc.

The Discrete Element Method (DEM) is one of the most popular methods for generating randomly packed beds (Eppinger et al.,

2011; Liu et al., 2021; Theuerkauf et al., 2006). The Discrete Element Method (DEM) is a numerical technique used to simulate the behavior of granular materials, such as particles in a packed bed (Li et al., 2022). In the context of packed bed generation, DEM represents each particle in the system as a discrete element with its position, size, shape, and material properties. These discrete elements interact with each other through contact forces, such as friction and repulsion, which govern their movement and behavior within the packed bed. DEM is widely used to generate packed beds for CFD modeling.

Lu and Nikrityuk proposed a fixed-bed reactor created via DEM (Yade) (Lu & Nikrityuk, 2018). The authors used Yade (Kozicki & Donze, 2009; Thabet & Straatman, 2018; Šmilauer et al., 2010) to generate fixed beds with spherical pellets. Yade is an open-source framework for discrete numerical models based on the discrete element method (DEM). Newton's second law of motion is used to track the particles in order to calculate the dynamics of pellets (Pouciau, 2006). Yade uses a soft-sphere collision model. It allows the rigid surfaces of particles to penetrate each other when they collide. More about Yade is described in these articles (Kozicki & Donze, 2009; Šmilauer et al., 2010).

Dixon and Partopour investigated alternative methods to DEM in the open-source software Blender (Dixon & Partopour, 2020; Partopour & Dixon, 2017). Blender based on rigid body dynamics (RBD) embedded in the Bullet Physics library (Featherstone, 2014; Yehia, 2022). The framework defines particles based on surface meshes, allowing free-form particles to be represented. The process of contact detection involves querying the proximity of vertices in objects. In order to optimize computation, it is common practice to partition the computational space into subspaces. These studies show that the technique can accurately reproduce the geometric packing properties of complexly shaped particles.

Moghaddam et al. (Moghaddam et al., 2018) proposed a packing algorithm based on RBD to simulate non-spherical pellets. The authors have used their own method based on RBD approach and realized an alternative method for transition between moving and stationary particles.

Karthik and Buwa (Karthik & Buwa, 2019) created a packed bed with six types of pellets via DesignModeler in Ansys (Fluent et al., 2011). This method was used to create a simple packed bed. This software has no special tools for packed bed generation and can be used only for the simplest packed beds. Therefore, this approach cannot be used in industry for the large-scale model.

Monfared et al. analyzed randomly packed beds generated via STAR-CCM+ code (Ghasemi et al., 2024). The authors used DEM to generate packed beds and compared void fraction with the packed beds generated via Blender and experiments. They reported a good correlation of porosity for the packed bed generated via STAR-CCM+ and experiments. The authors highlighted that they used commercial code.

Flaischlen and Wehinger conducted a comparative analysis of the packed bed models generated using STAR-CCM+ (Siemens) and Blender, as detailed in their study (Flaischlen & Wehinger, 2019). The research highlighted key distinctions between these two modeling approaches. In addition, the authors elucidated the surface structure representation of spherical particles. The surface of a spherical particle depicted as a collection of faces and edges, showcasing the non-uniform nature of the mesh structures. This non-uniformity poses challenges in computational mesh

generation for packed beds, particularly in the context of CFD modeling, potentially resulting in lower quality simulations.

Blender, a popular open-source software for packed bed generation, presents certain drawbacks that researchers should consider (Boccardo et al., 2015; Eghbalmanesh et al., 2024; Zhang et al., 2024). Some of these limitations include the need for program training, especially for beginners (Sakai et al., 2015). Additionally, Blender may result in uncontrolled contact points where element geometries intersect, potentially affecting the accuracy of the packed bed model. Another issue is the lack of support for CAD formats such as .stp and .step, which can restrict interoperability with other software tools (Agostini et al., 2022).

Furthermore, generating a large number of elements in Blender can demand high computer performance, causing a significant load on the system. Users also need to manually set both initial and boundary conditions, adding to the complexity of the modeling process. Calculating the number of elements in advance or selecting them manually is necessary, which can be time-consuming and prone to errors.

Moreover, there is a risk of bugs occurring, such as elements flying beyond the boundaries of the volume, potentially compromising the integrity of the simulation results (Dolamore et al., 2019). Lastly, the manual configuration of each element in Blender requires meticulous attention to detail and can be a labor-intensive task. Researchers should be aware of these limitations when considering Blender for packed bed modeling in computational fluid dynamics simulations.

The similar issues in the packed bed generation can occur in DEM (Yade). However, the packed beds generated via DEM (Yade) have fewer overlaps than Blender and require less computer performance than Blender. At the same time, Yade has some other disadvantages, e.g., beginners in modeling packed beds will need help to use this program because Yade runs on an Ubuntu system with a challenging interface. Another issue is that Yade cannot create standard-shaped cylinders. Therefore, using cylinders from Yade in CFD will be a difficult task. Yade will probably have a significant update in the future, and it will be better than it is right now.

The shortcomings of existing software programs prompted us to develop our own code to generate packed beds. The primary objective of this research article is to introduce and elucidate a novel Python-based code designed by authors specifically for the efficient generation of packed beds, with a focus on enhancing user convenience and ease of use, for example, within computational fluid dynamics (CFD) simulations. This newly developed code offers several key advantages that address the limitations observed in existing software tools, including:

- **High Generation Speed:** The code is optimized to generate packed bed geometries swiftly, enabling researchers to expedite the modeling process and reduce overall simulation time;
- **Solid State Format (.stp) Support:** Unlike certain software applications like Blender, this new code provides seamless support for Solid State format (.stp) files, enhancing interoperability with other CAD software tools commonly used in engineering and scientific research;
- **Complete Absence of Intersections of Elements:** By ensuring that there are no unintended intersections between elements in the generated packed bed model, the code enhances the

accuracy and reliability of the simulation results, thereby minimizing potential errors;

- **Quick and Easy Setup of Generation Parameters:** The user-friendly interface of the code facilitates the swift configuration of generation parameters, allowing researchers to customize the packed bed model according to their specific requirements with minimal effort;
- **Low CPU Load During Generation:** The code is designed to optimize computational efficiency, resulting in a reduced CPU load during the generation process. This not only enhances the performance of the software but also minimizes the strain on the system, making it suitable for running simulations on a variety of computing platforms.

By offering these key benefits, the new Python-based code presents a promising solution for researchers seeking a more efficient and user-friendly tool for generating packed bed models, e.g. in CFD simulations, ultimately contributing to advancements in the field of fluid dynamics research. This novel code is free for research community and shared as a supplementary material of the present paper.

2. Methodology

The main current approaches for generation of the packed bed models are based on rigid body dynamics (RBD) and Newton's laws (discrete element methods - DEM). This paper deals with the development and analysis of a novel code based on an analytical geometry approach for the packed bed generation.

The Rigid Body Dynamics (RBD) is integrated in Blender software where a single pellet is considered as a soft body. In RBD approach, the Newton-Euler equations are used to model the motion. The net force acting on the body, denoted as f , is equal to the change in translation momentum ($m \cdot v$) over time:

$$f = \frac{d(m \cdot v)}{dt} \quad (1)$$

To describe rotational motion, the variation of rotational momentum about time is represented by the net momentum equation τ :

$$\tau = \frac{d(I \cdot \omega)}{dt} \quad (2)$$

In order to calculate the tangential force components f_t and f_o , the force is determined based on the static friction factor μ and the normal force f_n :

$$f_t = \frac{-\mu \cdot f_n \cdot v_t}{\beta}; \quad f_o = \frac{-\mu \cdot f_n \cdot v_o}{\beta} \quad (3)$$

The tangential velocity components v_o and v_t at the contact point are additional factors used in calculating the sliding speed β :

$$\beta = \sqrt{u^2 + v^2} \quad (4)$$

The above provides an overview of the fundamental equations used in the RBD simulation method. For additional information on the theory of RBD simulation (Bender et al., 2014).

In the Discrete Element Method (DEM) (Cundall & Strack, 1979), integrated in Yade, the particles are presented as deformable pellets. Newton's second law is used as the basis. To determine the interactions between these pellets, the momentum balance equation is used:

$$m_p \frac{dv_p}{dt} = F_s + F_b. \quad (5)$$

where m_p and v_p are the pellets mass and velocity, t is the time.

In the simulation, surface forces F_s describe the forces acting on the pellet's surface. The body force F_b , consists of the contact forces F_c and the gravity force F_g :

$$F_b = F_g + F_c. \quad (6)$$

$$F_c = \sum_{\text{particles}} F_{\text{contact}} + \sum_{\text{walls}} F_{\text{contact}}. \quad (7)$$

The sum of the magnitudes of the normal and tangential forces and the unit normal vectors n and t are used to calculate the contact force between two spheres:

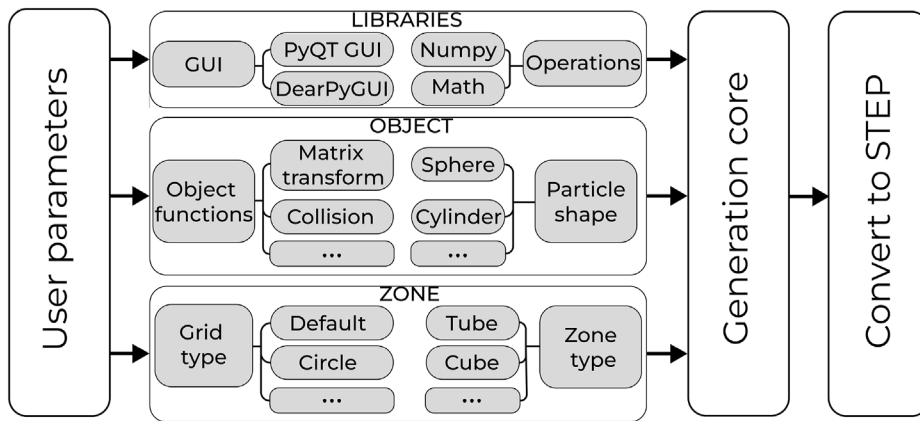


Fig. 1. Architecture of SAND code.

$$F_{contact} = F_n \cdot n + F_t \cdot t \quad (8)$$

Our novel Python-based SAND code for the packed bed generation is based on analytic geometry approach. This approach determines pellets as clouds of points and determines collision via inscribed and circumscribed circles. A detailed description of this approach is shown below.

2.1. Structure of SAND code

The software architecture is divided into three blocks, each responsible for solving specific tasks.

- The LIBRARIES block is responsible for connecting Python libraries. It is divided into two packages: one for GUI libraries and another for libraries used to implement mathematical operations.
- The OBJECT block is responsible for initializing objects and their functions. It is divided into initializes for different types of pellets and functions that implement spatial changes of pellets and their interactions with each other, such as collisions.
- The ZONE block is responsible for initializing the area filled with a packed bed. It is divided into two parts: defining an area's type and selecting a generation method.

Each block includes variables defined through the User Parameters block and set by the user through the Graphical User Interface (GUI). After receiving output data from the LIBRARIES, OBJECT and ZONE blocks, it is sent to the Generation CORE block as presented in Fig. 1.

Generation CORE is a block containing algorithms for packed bed generation based on the data from the previous blocks. The received data is used to generate output data, including the parameters for finding objects in space. The converter is then used to create the STEP format.

2.2. Generation algorithm

2.2.1. Object

Each element of a packed bed, except the spheres, is a polygonal object consisting of vertices, edges and faces. This code creates an element using an array of vertices in a four-dimensional format and defining surfaces by 4 points.

The data below describe a cylinder inscribed in a 1 mm cube.

```
cylinder_vertex_init6 = (
(0.0, 1.0, 0.5, 1),
(0.866025, 0.5, 0.5, 1),
(0.866025, -0.5, 0.5, 1),
(0, -1.0, 0.5, 1),
(-0.866025, -0.5, 0.5, 1),
(-0.866025, 0.5, 0.5, 1),

(0.0, 1.0, -0.5, 1),
(0.866025, 0.5, -0.5, 1),
(0.866025, -0.5, -0.5, 1),
(0, -1.0, -0.5, 1),
(-0.866025, -0.5, -0.5, 1),
(-0.866025, 0.5, -0.5, 1)
)
cylinder_faces_init6 = (
(0 ,1, 7, 6),
(1, 2, 8, 7),
(2, 3, 9, 8),
(3, 4, 10, 9),
(4, 5, 11, 10),
(5, 0, 6, 11),

(5, 4, 3, 2, 1, 0),
(6, 7 ,8, 9, 10, 11)
)
```

Further an object class was created using these arrays. In this example, a cylinder is a 6-angle. This class is used to initialize element parameters such as radius, height, position in space and rotation angles. Also, inside the class definition, there are auxiliary methods required to determine the interaction of elements with each other as presented below.

```

class Cylinder6:
    radius = get_value_parameters()["cylinder_radius"] * 1.155
    height = get_value_parameters()["cylinder_height"] + 0.05

    #initialization cylinder vertex (transformation scale)
    vertex_init = np.array(cylinder_vertex_init6) @ scale (radius, radius, height)

    def __init__(self, x: int | float, y: int | float, z: int | float, ax: int | float
    = 0, az: int | float = 0):
        self.x, self.y, self.z = x, y, z
        self.ax, self.az = ax, az
        self.pos = np.array((self.x, self.y, self.z, 1))

        self.vertex = Cylinder6.vertex_init @ rotated_x(ax)
        self.vertex = self.vertex @ rotated_z(az)
        self.vertex = self.vertex @ move(self.x, self.y, self.z)
    def faces(self) -> tuple:
        """Return tuple, where faces replace vertex on ID"""
        return tuple(tuple(self.vertex[j] for j in i) for i in cylinder_faces_init6)

```

A matrix section of Python-based code considers the transformation matrices of three-dimensional objects in space, precisely the scale, rotate, and move functions. These matrices are responsible for zooming, rotating, and moving pellets. Transformation

matrices can be used to manipulate the position of pellets in space by multiplying them with arrays of object vertices. The code for this section is shown below.

```

# basic functions
def scale(sx: int | float, sy: int | float, sz: int | float) -> np.ndarray:
    """Scale matrix"""
    return np.array([
        [sx, 0, 0, 0],
        [0, sy, 0, 0],
        [0, 0, sz, 0],
        [0, 0, 0, 1]
    ])

def move(x: int | float, y: int | float, z: int | float) -> np.ndarray:
    """Move matrix"""
    return np.array([
        [1, 0, 0, x],
        [0, 1, 0, y],
        [0, 0, 1, z],
        [0, 0, 0, 1]
    ]).T

def rotated_x(a: int | float) -> np.ndarray:
    """Rotate x-axis matrix"""
    rad = np.radians(a % 360)
    return np.array([
        [1, 0, 0, 0],

        [0, np.cos(rad), -np.sin(rad), 0],
        [0, np.sin(rad), np.cos(rad), 0],
        [0, 0, 0, 1]
    ])

```

The code includes four different types of cylinders: cylinder-6, cylinder-8, cylinder-12, and cylinder-24. The numerical suffix in each type denotes the number of edges that form the circumference of the cylinder. This distinction is crucial for refining the generation process to strike a balance between the porosity of the packed bed and the time taken for generation. Opting for a higher edge count results in a more realistic density but also increases the generation time. Therefore, selecting the appropriate cylinder type is essential to achieve the desired level of detail while managing computational efficiency.

Consequently, the development of a specialized library was undertaken to facilitate the creation and transformation of pellets with the flexibility for precise adjustments and refinements. This library serves as a valuable tool for streamlining the generation process while ensuring meticulous control over the resulting output.

2.2.2. Collision

In this subsection, the algorithm designed to detect collisions between pellets is described. The algorithm's structure varies depending on the shape of the pellet, necessitating distinct approaches for spherical pellets and more intricate shapes.

When dealing with spherical pellets, collision detection is based on calculating the distance between their geometric centers. If this distance exceeds the sum of their radii, it signifies that the elements are not in contact. Conversely, if the distance is less than or equal to the sum of their radii, a collision is detected between the pellets. This straightforward method provides a clear criterion for determining interactions between spherical pellets and serves as a foundational principle in collision detection algorithms.

Below is the code that implements the collision for elements of type “ball”.

```
def collision_global_ball2(objs, obj_a):
    for obj_b in objs:
        if dist(obj_a.pos, obj_b.pos) < obj_a.radius * 2:

            return False
        else:
            return True

def collision_ball(ball1, ball2) -> bool:
    return True if dist(ball1.pos, ball2.pos) > ball1.radius * 2 else False

def collision_global_ball(objs: tuple | list, obj, tube_radius: int | float) -> bool:
    return all(
        tuple(
            dist(obj.pos, np.array((0, 0, obj.pos[2], 1))) < tube_radius - obj.radius
            and collision_ball(i, obj) for i in objs
        )
    )
```

A more intricate collision detection algorithm is required for cylindrical pellets due to their rotational nature in space. This advanced algorithm is implemented utilizing a combination of the discrete element method (DEM) (Munjiza, 2004) and the Gilbert-Johnson-Keerthi (GJK) algorithm (Ong & Gilbert, 2001), drawing on the strengths of both approaches. The DEM provides a robust framework for simulating interactions between discrete elements,

while the GJK algorithm excels in efficiently detecting collisions between complex shapes.

In our algorithm, the surface of the cylindrical pellets is defined using a surface definition method that derives the surface equation from three key points. By capturing the essential geometric properties of the cylindrical shape through these points, the algorithm can accurately assess collisions and interactions involving these more intricate pellet structures. This approach enhances the accuracy and reliability of collision detection for cylindrical pellets within the simulation environment.

2.2.3. Generation domain and grid

The generation domain within our code defaults to a cylindrical pipe, with parameters for radius and length. However, we are actively working on expanding this feature to include additional types of domains in the upcoming version of our code.

Once the boundary conditions of the generation domain are defined, the subsequent step involves creating a grid that serves as the foundation for constructing the packed bed. The generation area is populated with a grid of points, each of which is evaluated for its potential to accommodate an element based on specified size and rotation angle parameters. Our code supports the utilization of two distinct grid types, each tailored to suit different simulation requirements.

- Standard grid:

The standard grid is a collection of points arranged in ascending order along the Z axis as presented in Fig. 2(a). The user can select the step between the points and customize the generation accuracy by manually setting the distance between points (s) as shown in Fig. 2(b).

- Circular grid:

The circular grid is a collection of points arranged in a circle with decreasing radius according to Fig. 3(a). The user can adjust the distance between the circles and the points (r and s, respectively) as illustrated in Fig. 3(b). This grid is recommended for generating in the pipe area.

```

def ball_gen(grid: tuple | list | np.ndarray):
    objs = []
    for point in grid:
        ball = Ball(point[0], point[1], point[2])
        ball.radius = get_value_parameters()["ball_radius"]
        if collision_global_ball2(objs, ball):
            objs.append(ball)
            xa.append(ball.x)
            za.append(ball.z)
            ya.append(ball.y)
            dpd.set_value("series1", [ya, za])
            dpd.set_value("series2", [xa, za])
    xa.clear()
    za.clear()
    ya.clear()
    return objs

```

2.2.4. Generation method

After initializing the object and establishing the grid area, the subsequent step involves initiating the generation process. The intricacy of the generation methods employed will vary based on the chosen area and the complexity of the pellets being generated. Our program currently incorporates cylinder and sphere generation methods, with plans to expand to include additional shapes in future updates.

Example of sphere generation.

This algorithm oversamples a grid point and checks for any detected contact points. If an contact point is detected, no new object is generated. This process is repeated for each subsequent element, resulting in a list of elements with size and position parameters.

The cylinder generation algorithm uses a similar method but requires a proper function to determine turns at specific positions in the domain. To implement this method of rotations, a pseudo-random search of the positions is used, along with some probabilistic functions, to select the correct angle more accurately.

The list is sent to the primary generation function, where user parameters are defined, and the resulting layer is saved in *.stp* format.

```

def generation_start(do_save = False):
    match get_value_parameters()["grid_type"]:
        case 'default':
            main_grid = default_grid()
        case "circle":
            main_grid = circle_grid()
        case _:
            main_grid = None
    if get_value_parameters()["object_type"] in ('cylinder6', 'cylinder8', 'cylinder12',
        'cylinder24'):
        start = time()
        elements = cylinder_gen(get_value_parameters()["object_type"], main_grid)
        dpd.set_value('time_gen', f"Time generation: {(time() - start):0.3f}s")
        if do_save:
            save_cylinder_stp(elements, "Fixed_bed(" +
                str(datetime.datetime.now()).replace('-', '_').replace(" ",
                    '#')[0:19].replace(':', '_') + ")" + "_" + str(randint(11111111, 99999999)),
                get_value_parameters()["save_file_path"])

    if get_value_parameters()["object_type"] == "ball":
        start = time()
        elements = ball_gen(main_grid)
        dpd.set_value('time_gen', f"Time generation: {(time() - start):0.3f} s")
        if do_save:
            save_ball_stp(elements, "Fixed_bed(" +
                str(datetime.datetime.now()).replace('-', '_').replace(" ",
                    '#')[0:19].replace(':', '_') + ")" + "_" + str(randint(11111111,
                    99999999)), get_value_parameters()["save_file_path"])

```

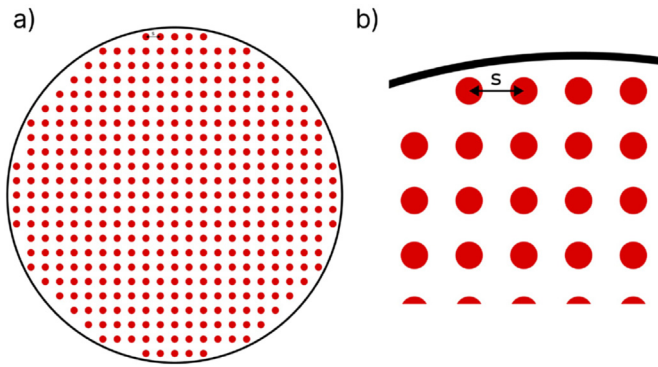


Fig. 2. Generation scheme of a standard grid.

3. Results and discussions

In order to verify the novel Python-based code (SAND) for the generation of a packed bed, several comparisons were carried out with existing packed bed generation programs (DEM – Yade, RBD – Blender) and with experiments (Papkov et al., 2024).

Filling strategies of generation for all methods are presented in Fig. 4. Two filling generation strategies were used in Blender. The first strategy is a simple method in Blender, described in detail in the articles (Boccardo et al., 2015; Partopour & Dixon, 2017). The second is the author's idea based on the hitbox about pellets

(Fig. 4(d–e)). The packed beds were created via various methods with equal geometric parameters for the comparison.

In the present work, the following case studies are carried out and compared.

- All generation methods were compared with each other at constant height. However, no comparison was made between cylinders and DEM (Yade). This is because Yade cannot create a standard shape-form cylinder.
- The packed bed generated by SAND with cylindrical and spherical pellets at constant radius compared with the experiment described in the previous article.
- The radial porosity of a packed bed with spherical and cylindrical pellets was measured. This radial porosity for spherical pellets is compared against DEM (Yade) and RBD (Blender).
- The computational time for the packed bed generation filled with either cylinders and spheres was compared for Blender, Yade, SAND.

3.1. Spheres

Fig. 5 illustrates the varying number of pellets within the packed bed, generated through various methods. The number of pellets ranges from 150 to 3500. The number of pellets in the packed bed models was compared with actual number of pellets. The actual measurements were performed for the packed bed radius up to 150 mm. Initially, no discernible distinctions were observed between the Discrete Element Method (DEM) implemented in Yade, the Rigid Body Dynamics (RBD) utilized in Blender, and the SAND method at smaller D/d ratios, where D is the packed bed diameter and d is the particle diameter. However, as the D/d ratio increases, disparities between the methods become more notable. This discrepancy can be attributed to potential particle overlap during the generation of the packed bed using Blender and Yade.

The average deviation in the number of pellets between experimental data and the packed bed generated via the SAND approach amounts to 0.4% for packed bed radii ranging from 50 to 125 mm, and approximately 1.2% for radii of 125–250 mm. Notably, most numerical simulations involving packed beds focus on diameters within the 50–100 mm range. Consequently, it is deduced that the SAND code exhibits a strong correlation in pellet count between experimental data and our model. Furthermore, no significant differences were detected between the two methods in

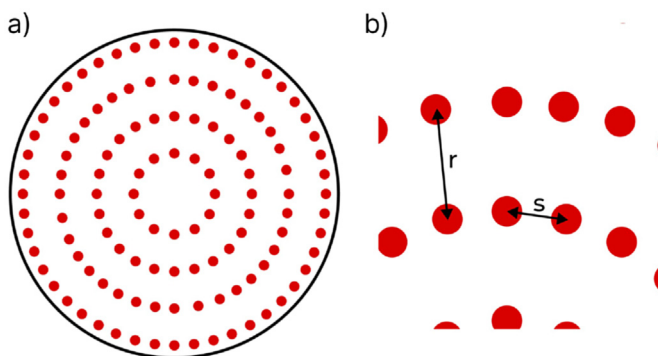


Fig. 3. Generation scheme of a circular grid.

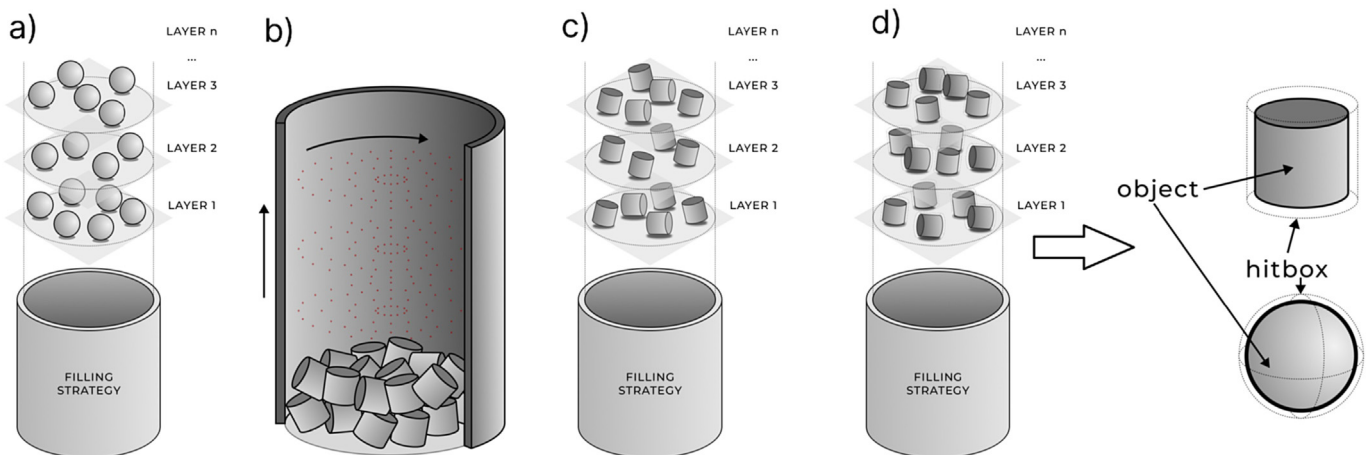


Fig. 4. Filling strategy for: (a) DEM (Yade), (b) SAND, (c) 1st method in Blender, (d) and (e) 2nd method in Blender.

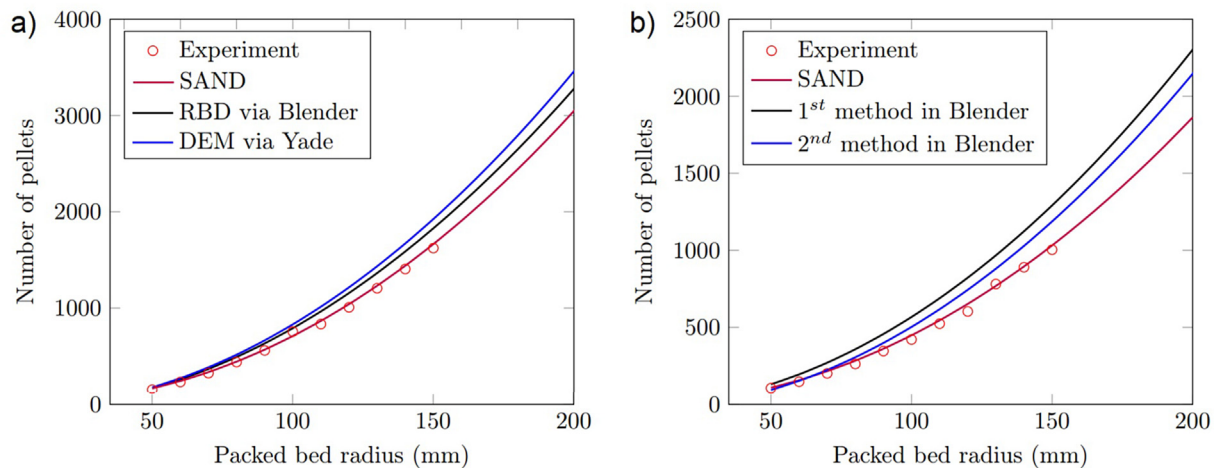


Fig. 5. Comparison of the number of pellets between SAND, DEM (Yade), 1st and 2nd method in Blender for: (a) spheres and (b) cylinders at constant height ($H = 200$ mm).

Blender as described earlier, leading to the selection of the first method in Blender for subsequent analyses.

SAND code avoids overlapping in a packed bed while for the packed bed generated via Blender and Yade the overlaps can be observed as shown in Fig. 6. The red points represent overlaps in packed beds. The packed beds generated via Blender and Yade have a significant number of overlaps because these tools are based on gravity. At the same time, SAND demonstrates an actual axial porosity without overlaps between particles.

Fig. 7 illustrates the packed beds created using Rigid Body Dynamics (RBD) in Blender, the SAND method, and DEM in Yade. SEND-DEM code is adopted for generation of packed bed for CFD-modelling. This code includes a tool to regulate the distance between particles, crucial for preventing errors in the computational

mesh in CFD modeling (Dixon et al., 2013). Consequently, there will be no need for geometric downscaling of the computational domain for CFD modeling.

Fig. 8 shows the results of comparison of the actual number of the pellets with the packed bed generated by SAND. Fig. 8 shows a good correlation between spherical types of pellets. Fig. 8 shows that large empty spaces between layers were not detected between spherical pellets. Also, SAND demonstrates a good correlation with the experiment at low heights of packed beds.

3.2. Facets

The packed bed is composed of individual pellets. Fig. 9 shows the spherical pellets generated using Blender (a), Yade (b), and

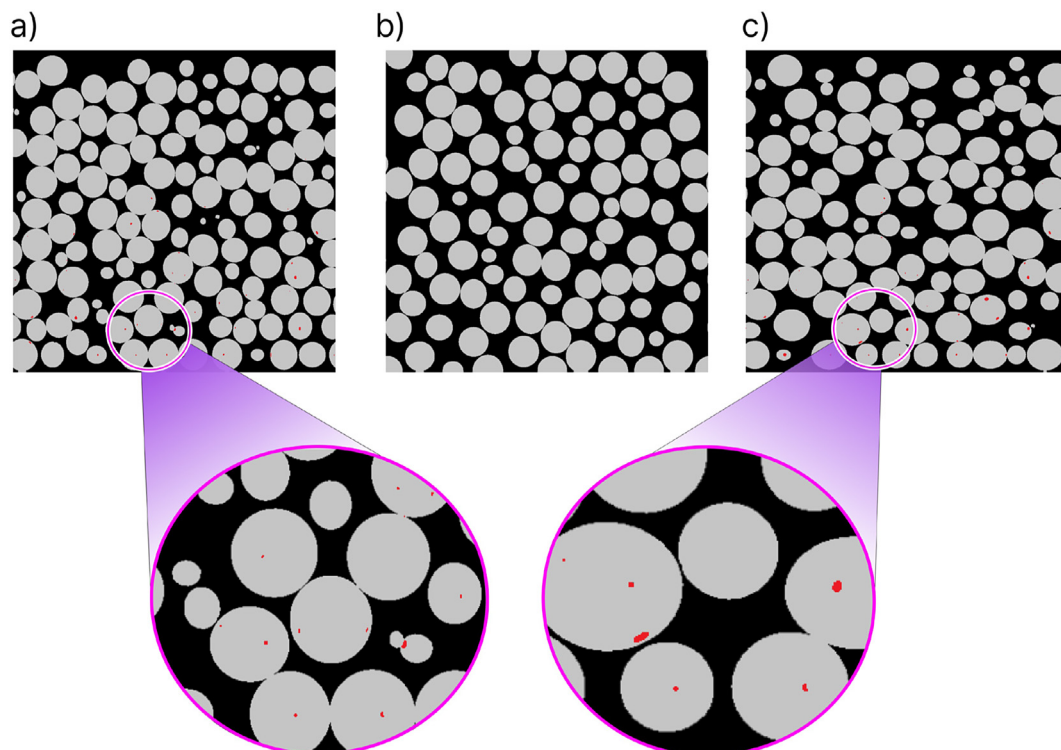


Fig. 6. Axial plane view for the packed beds generated via (a) RBD (Blender), (b) SAND and (c) DEM (Yade).

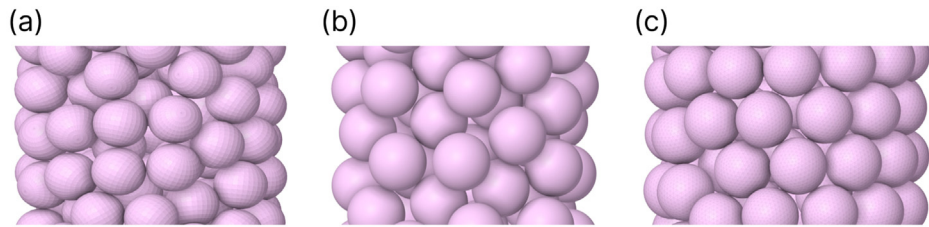


Fig. 7. The packed beds generated via (a) RBD (Blender), (b) SAND and (c) DEM (Yade).

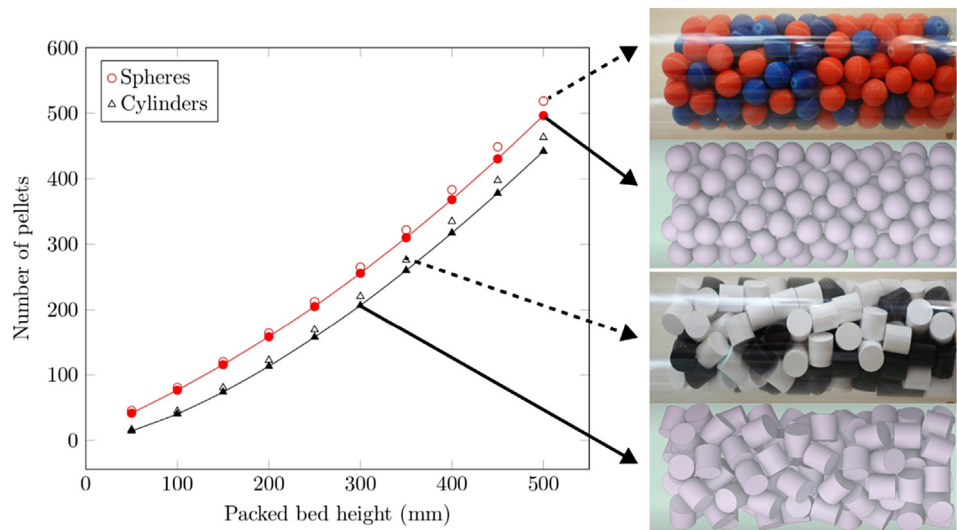


Fig. 8. Comparison of the number of pellets between SAND and experiment for a packed bed with radius of 50 mm.

SAND (c). In Blender and Yade, the pellets are represented by multiple facets. Additionally, the size of facets in the spheres generated by Blender varies, which can pose challenges in computational mesh generation for CFD modeling. This variation can result in a non-uniform mesh structure and potentially introduce errors. To mitigate these issues, many researchers opt to downscale the pellets in the computational domain. By doing so, they aim to ensure a stable solution in CFD simulations.

SAND code is designed to create spherical pellets as a single facet and cylindrical pellets as three facets. The underlying principle of pellet generation in SAND revolves around minimizing facets to streamline the process. The pellets are generated in a standard format, such as *.step*, within this code.

On the other hand, Blender has the capability to produce geometry in *.stl* format, while DEM software like Yade does not have direct support for generating pellets in *.stl* or *.step* formats. In Yade, a point cloud is exported, and this cloud is subsequently utilized to

construct a pellet. This distinction in file formats and generation methods highlights the diverse approaches employed by different software tools in creating and manipulating geometric shapes for simulations and modeling purposes.

3.3. Cylinders

Cylindrical pellets are commonly used in packed beds, but generating a packed bed filled with cylinders can be challenging due to issues like those encountered with spherical particles, as well as significant particle overlap. The SAND code offers a solution to generate packed beds without particle overlap.

Fig. 5 illustrates packed beds created using Blender (a) and SAND (b). The parameters of packed beds filled with cylinders show good agreement with experimental data, especially at low D/d ratios as shown in Figs. 8 and 5. However, deviations are more pronounced at high D/d ratios compared to the spherical case,

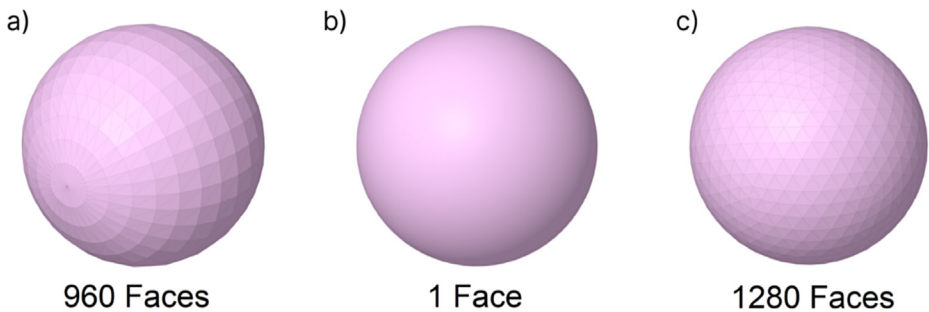


Fig. 9. A surface structure of a spherical particle: (a) RBD (Blender), (b) SAND, (c) DEM (Yade).

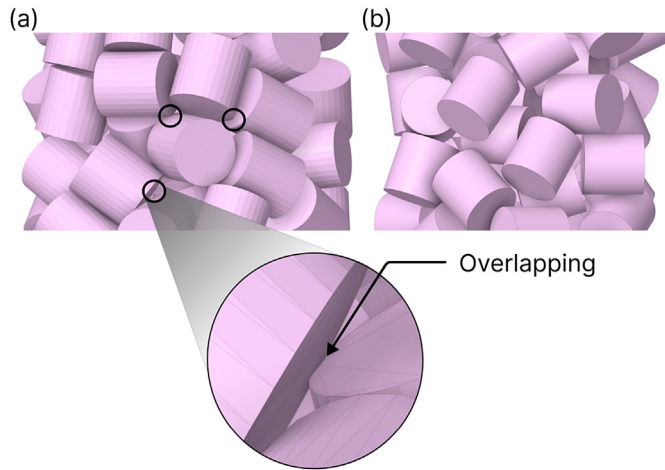


Fig. 10. The packed beds filled with cylinders generated via (a) RBD (Blender) and (b) SAND.

primarily due to the increased degrees of freedom for the cylinders comparing to the spheres. Cylinders can rotate in three coordinates, necessitating a higher number of iterations.

Despite the simple shape of cylinders, generating packed beds filled with cylinders poses modeling challenges (Dorai et al., 2015; Niegodajew et al., 2022). Contact points between cylinders present a major issue for the packed bed models, as shown in Fig. 10. Downsizing each cylinder can resolve this problem but may lead to increased bed porosity and discrepancies between modeled and real packed beds. Higher porosity can significantly impact pressure drop and other parameters. In contrast, SAND avoids overlaps by utilizing analytical geometry, ensuring accurate and efficient pellet generation.

3.4. Radial porosity

The radial porosity of both spherical and cylindrical pellets was thoroughly investigated in our study. We examined the radial porosity of a packed bed filled with spherical pellets and compared it to the radial porosity of packed beds generated using DEM (Yade) and RBD (Blender), as illustrated in Fig. 11(a). Additionally, we analyzed the radial porosity of a packed bed containing cylindrical pellets and compared it to the radial porosity of a packed bed generated through RBD (Blender), as depicted in Fig. 11(b).

Our findings revealed that the results obtained using the SAND code were in good agreement with experimental data and the

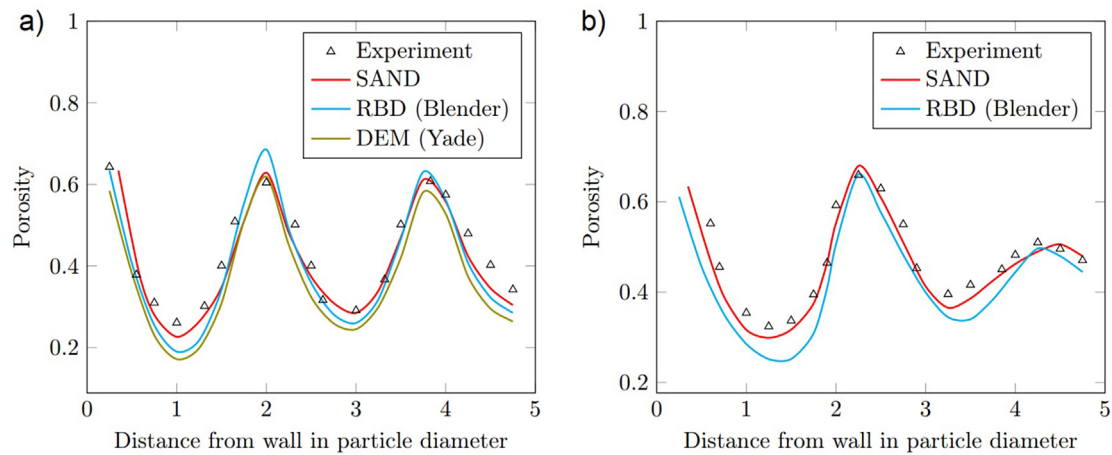


Fig. 11. Radial porosity for RBD (Blender), DEM (Yade) and SAND for (a) spherical pellets and (b) cylindrical pellets.

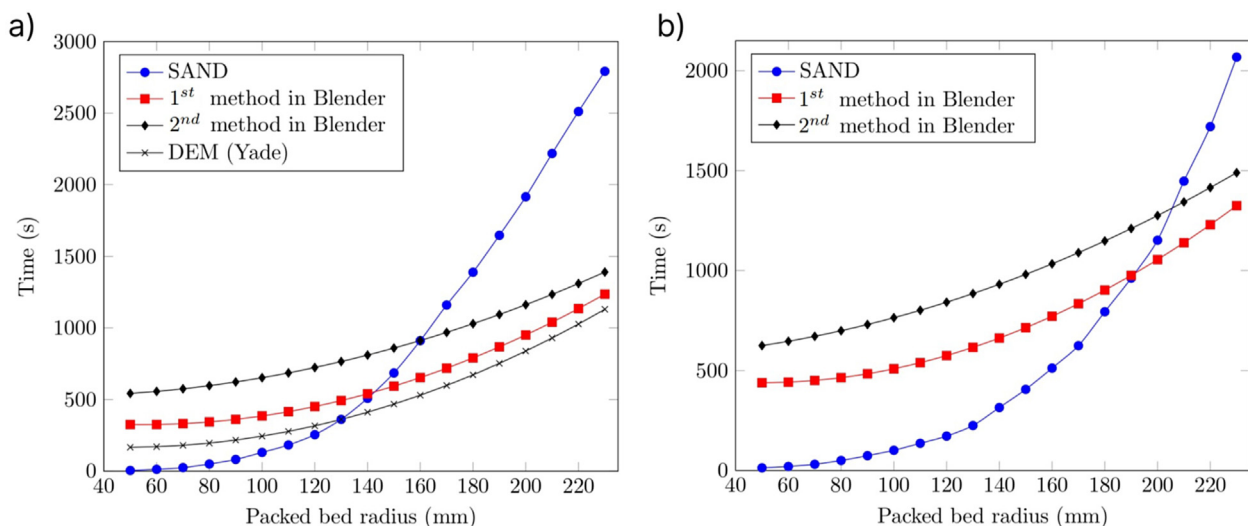


Fig. 12. The computational time for generation of the packed beds with length of 200 mm filled with (a) spherical ($d = 20$ mm) and (b) cylindrical ($d = 20$ mm, $h = 20$ mm) pellets.

Table 1
Summary of the codes for a packed bed generation.

Generation features	RBD (Blender)	DEM (Yade)	SAND
High Generation speed	—	—	+
Solid format support	—	—	+
Quick and easy setup	—	—	+
Low CPU Load during generation	—	—	+
No overlaps	^a —	—	+
Training in the program is not a requirement	—	—	+
Friendly user interface	—	—	+
Both initial and boundary conditions must not be set as a requirement	—	—	+
It is not necessary to determine the number of elements beforehand	—	—	+
Fully manual customization of each element.	+	—	—
Errors occur during a packed bed generation	+	+	—
High density of the packed bed	+	+	—

^a Overlaps can be removed using the 2nd method in Blender described above.

radial porosity data from the packed beds generated via RBD (Blender) and DEM (Yade). This alignment underscores the reliability and accuracy of the SAND code in predicting the radial porosity of packed beds filled with both spherical and cylindrical pellets.

3.5. Computational time

Packed bed generation involves the intricate assembly of hundreds or even thousands of individual elements governed by various physical laws. Consequently, the computational time required for generating packed bed models can be substantial and escalates as the number of elements increases. In the SAND code, our focus has been on optimizing computational efficiency to swiftly obtain packed bed models. Particularly, we directed our efforts towards generating packed beds with diameters up to 300 mm, a common dimension in numerical simulations.

To evaluate and compare the computational efficiency of packed bed generation, we conducted tests using Blender, Yade, and SAND on a PC equipped with an Intel(R) Core(TM) i9-7980XE processor. The length of the packed beds in all cases was set at 200 mm, and they were filled either with spherical particles of 10 mm radius or cylindrical particles with a radius of 10 mm and a height of 20 mm.

The results of the computational time analysis are presented in Fig. 12.

From the data in Fig. 12, it is evident that for packed beds with radii up to 125 mm, the SAND code demonstrates the shortest computational time. Notably, radii up to 100 mm are commonly used in numerical simulations of packed beds. The graph also illustrates that SAND can generate a packed bed with a 5 mm radius in just a few seconds. However, as the radius of the packed bed increases, the computational time for SAND code significantly rises.

One key factor contributing to this difference in computational time is that SAND currently operates with a single logical process, while DEM and RBD utilize multiple logical processes. In future updates, we plan to enhance the efficiency of SAND by increasing the number of logical processes to reduce the computational time required for generating packed beds with larger radii.

3.6. Summary for the codes

Table 1 presents a summary of the considered codes (Blender, Yade, SAND) for packed bed generation, highlighting their main advantages and disadvantages. It is important to emphasize that the novel code SAND demonstrates better performances compared to Blender and Yade in several key aspects. Notably, SAND excels in

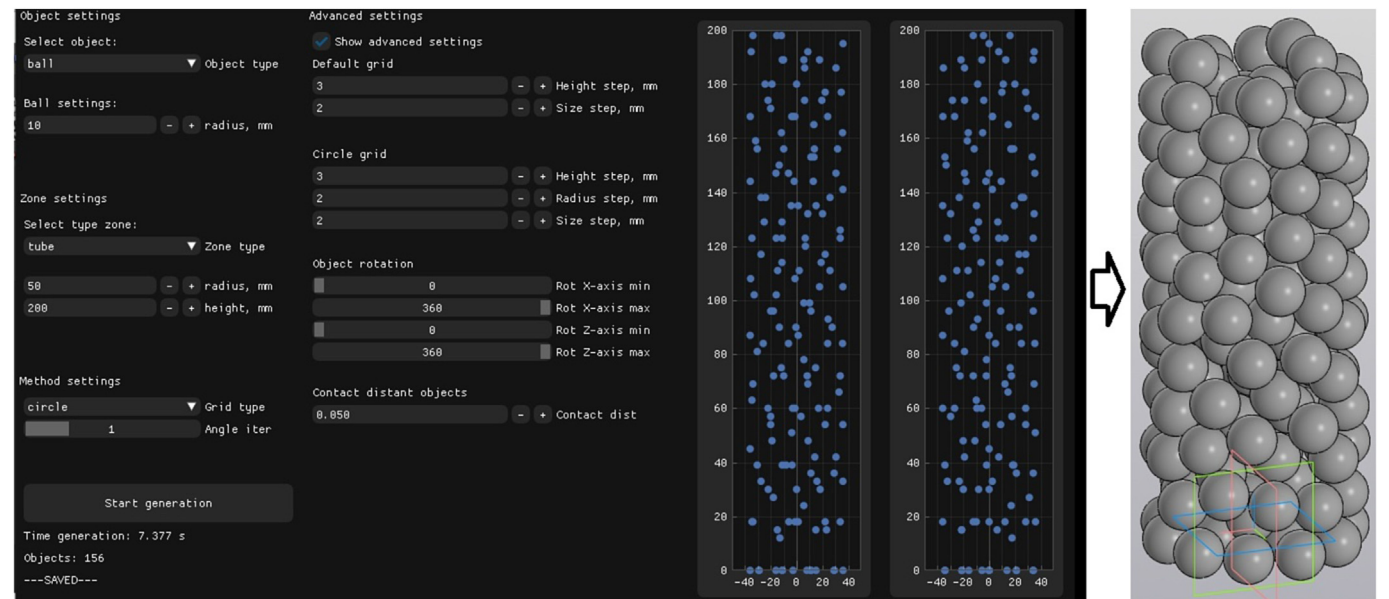


Fig. 13. Interface of SAND code for the packed bed generation.

the high-speed generation of packed beds, particularly for low D/d ratios, supports solid formats for packed bed CAD models, ensures no overlaps between pellets, and exhibits error-free generation processes. Furthermore, SAND code has a user-friendly interface and does not require boundary or initial conditions, enhancing its ease of use and efficiency.

On the flip side, the SAND code exhibits higher deviations in the porosity of packed beds filled with cylindrical pellets when compared to those generated using Blender. This variation underscores the importance of considering different factors and configurations when assessing the accuracy and reliability of packed bed generation codes.

A workbench interface of the SAND code and view of packed bed model is presented in Fig. 13. Fig. 13 shows the efficient generation process, with a total time of 7.737 s required to create a packed bed with a radius of 50 mm and a height of 200 mm, filled with spherical pellets having a radius of 10 mm. The packed bed contains a total of 156 pellets, showcasing the code's capability to quickly and accurately generate complex configurations. This visual representation provides a clear insight into the user-friendly interface and the speed at which SAND can create packed bed geometries, making it a valuable tool for researchers and engineers in various fields.

4. Conclusion

In this work, a novel Python-based code for the generation of randomly packed beds filled with various pellets is developed and analyzed. The currently widely used codes such as Blender and Yade are based on rigid body dynamics and Newton's laws, respectively. The distinguished feature of the novel SAND code lies in the use of an analytic geometry approach.

The parameters of the packed bed generated via the SAND code were compared with experimental data as well as with the parameters of the packed beds generated via RBD (Blender) and DEM (Yade) approaches. The total number of pellets in the packed bed and radial porosity were compared. The average deviation in the number of pellets between experimental data and the packed bed generated via the SAND approach amounts to 0.4% for packed bed radii ranging from 50 to 125 mm, and approximately 1.2% for radii of 125–250 mm. The radial porosity for the packed bed generated via the SAND code is also in good agreement with experiments and other approaches.

The main advantages of the SAND code are as follows: high-speed generation of packed beds, particularly for low D/d ratios, support for solid formats for packed bed CAD models, ensuring no overlaps between pellets, and exhibiting error-free generation processes. Furthermore, the SAND code has a user-friendly interface and does not require boundary or initial conditions, enhancing its ease of use and efficiency. The SAND code is attached to this paper as supplementary materials and can be freely used by engineers and scientists for packed bed generation.

This manuscript presents the first version of the SAND code. A basic methodology and algorithm of the code are shown. At this version, the code can generate the packed beds filled with spheres and cylinders. Also, the applicable range for packed bed generation for particle to tube diameter ratio currently begins from $D/d = 2$ to $D/d = 10$; minimum packed bed height is 1 mm and maximum height depends on number of pellets; the maximum number of pellets is dependent on computational power. No limits for maximal number of particles are set. However, we plan to improve the features of the SAND code in the future. At the next versions, various particle shapes (Raschig rings, 7-hole cylinders, trilobites, etc.) and boundaries will be available. Furthermore, after these updates, an option to set arbitrary shapes and boundaries will be

added. In addition, the SAND will account real particle properties (friction, deformation, etc.) for packed bed generation.

CRediT authorship contribution statement

Nikita Shadymov: Visualization, Validation, Software, Methodology, Investigation. **Viacheslav Papkov:** Writing – original draft, Visualization, Validation, Investigation. **Dmitry Pashchenko:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Supplementary materials

The SAND code for packed bed generation has been uploaded to Mendeley Data and is available for free use by the scientific community: <https://doi.org/10.17632/m6pdjy4bx5.1>.

References

- Afandizadeh, S., & Foumeny, E. (2001). Design of packed bed reactors: Guides to catalyst shape, size, and loading selection. *Applied Thermal Engineering*, 21(6), 669–682.
- Agostini, E., Boccardo, G., & Marchisio, D. (2022). An open-source workflow for open-cell foams modelling: Geometry generation and cfd simulations for momentum and mass transport. *Chemical Engineering Science*, 255, Article 117583.
- Bender, J., Erleben, K., & Trinkle, J. (2014). Interactive simulation of rigid body dynamics in computer graphics. *Computer Graphics Forum*, 33(1), 246–270. plus 0.5em minus 0.4emWiley Online Library.
- Boccardo, G., Augier, F., Haroun, Y., Ferré, D., & Marchisio, D. L. (2015). Validation of a novel open-source work-flow for the simulation of packed-bed reactors. *Chemical Engineering Journal*, 279, 809–820.
- Costello, M., Gatto, S., & Sahu, J. (2007). Using cfd/rbd results to generate aerodynamic models for projectile flight simulation. In *AIAA atmospheric flight mechanics conference and exhibit* (p. 6582).
- Cundall, P. A., & Strack, O. D. (1979). A discrete numerical model for granular assemblies. *Geotechnique*, 29(1), 47–65.
- Diersch, H.-J., & Kolditz, O. (2002). Variable-density flow and transport in porous media: Approaches and challenges. *Advances in Water Resources*, 25(8–12), 899–944.
- Dixon, A. G., Nijemeisland, M., & Stitt, E. H. (2013). Systematic mesh development for 3d cfd simulation of fixed beds: Contact points study. *Computers & Chemical Engineering*, 48, 135–153.
- Dixon, A. G., & Partopour, B. (2020). Computational fluid dynamics for fixed bed reactor design. *Annual Review of Chemical and Biomolecular Engineering*, 11, 109–130.
- Dolamore, F., Dimartino, S., & Fee, C. J. (2019). Numerical elucidation of flow and dispersion in ordered packed beds: Nonspherical polygons and the effect of particle overlap on chromatographic performance. *Analytical Chemistry*, 91(23), 15 009–015 016.
- Dorai, F., Teixeira, C. M., Rolland, M., Climent, E., Marcoux, M., & Wachs, A. (2015). Fully resolved simulations of the flow through a packed bed of cylinders: Effect of size distribution. *Chemical Engineering Science*, 129, 180–192.
- Eghbalmanesh, A., Fathiganjehlou, A., Rieder, D. R., Hoogendoorn, M., Miloshevska, M., Baltussen, M. W., Peters, E., Buist, K. A., & Kuipers, J. (2024). Cfd-validated pore network modeling of packed beds of non-spherical particle. *Chemical Engineering Science*, 283, Article 119396.
- Eppinger, T., Seidler, K., & Kraume, M. (2011). Dem-cfd simulations of fixed bed reactors with small tube to particle diameter ratios. *Chemical Engineering Journal*, 166(1), 324–331.
- Featherstone, R. (2014). *Rigid body dynamics algorithms. plus 0.5em minus 0.4emSpringer*.
- Flaischlen, S., & Wehinger, G. D. (2019). Synthetic packed-bed generation for cfd simulations: Blender vs. star-ccm+. *ChemEngineering*, 3(2), 52.
- Fluent, A., et al. (2011). *Ansys fluent theory guide* (Vol. 15317, pp. 724–746). USA: Ansys Inc.
- Gaber, C., Demuth, M., Prieler, R., Schluckner, C., & Hochenauer, C. (2018). An experimental study of a thermochemical regeneration waste heat recovery process using a reformer unit. *Energy*, 155, 381–391.
- Ghasemi Monfared, Z., Hellström, J. G. I., & Umeki, K. (2024). The impact of discrete element method parameters on realistic representation of spherical particles in a packed bed. *Processes*, 12(1), 183.
- Habib, M. A., Harale, A., Paglieri, S., Alrashed, F. S., Al-Sayoud, A., Rao, M. V., Nemitalah, M. A., Hossain, S., Hussien, M., Ali, A., et al. (2021). Palladium-alloy membrane reactors for fuel reforming and hydrogen production: A review. *Energy & Fuels*, 35(7), 5558–5593.

- Jafari, A., Zamankhan, P., Mousavi, S., & Pietarinen, K. (2008). Modeling and cfd simulation of flow behavior and dispersivity through randomly packed bed reactors. *Chemical Engineering Journal*, 144(3), 476–482.
- Jurtz, N., Kraume, M., & Wehinger, G. D. (2019). Advances in fixed-bed reactor modeling using particle-resolved computational fluid dynamics (cfd). *Reviews in Chemical Engineering*, 35(2), 139–190.
- Karpilov, I., & Pashchenko, D. (2023). Steam methane reforming over a preheated packed bed: Heat and mass transfer in a transient process. *Thermal Science and Engineering Progress*, 42, Article 101868.
- Karthik, G., & Buwa, V. V. (2019). Effect of particle shape on catalyst deactivation using particle-resolved cfd simulations. *Chemical Engineering Journal*, 377, Article 120164. iSCRE 25 Special Issue: Bridging Science and Technology. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1385894718320539>.
- Kloss, C., Goniva, C., Hager, A., Amberger, S., & Pirker, S. (2012). Models, algorithms and validation for opensource dem and cfd–dem. *Progress in Computational Fluid Dynamics, an International Journal*, 12(2–3), 140–152.
- Kozicki, J., & Donze, F. V. (2009). Yade-open dem: An open-source software using a discrete element method to simulate granular material. *Engineering Computations*, 26(7), 786–805.
- Krishnamurthy, R., & Taylor, R. (1985). Simulation of packed distillation and absorption columns. *Industrial and Engineering Chemistry Process Design and Development*, 24(3), 513–524.
- Li, Q., Tang, R., Wang, S., & Zou, Z. (2022). A coupled les-lbm-imb-dem modeling for evaluating pressure drop of a heterogeneous alternating-layer packed bed. *Chemical Engineering Journal*, 433, Article 133529 [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1385894721051032>.
- Liu, X., Gui, N., Yang, X., Tu, J., & Jiang, S. (2021). A new discrete element-embedded finite element method for transient deformation, movement and heat transfer in packed bed. *International Journal of Heat and Mass Transfer*, 165, Article 120714.
- Lu, Y. R., & Nikrityuk, P. (2018). A fixed-bed reactor for energy storage in chemicals (e2c): Proof of concept. *Applied Energy*, 228, 593–607 [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306261918309784>.
- Moghaddam, E. M., Foumeny, E. A., Stankiewicz, A. I., & Padding, J. T. (2018). Rigid body dynamics algorithm for modeling random packing structures of nonspherical and nonconvex pellets. *Industrial & Engineering Chemistry Research*, 57(44), 14988–15007.
- Mokheimer, E. M. A., Shakeel, M. R., Harale, A., Paglieri, S., & Mansour, R. B. (2024). Fuel reforming processes for hydrogen production. *Fuel*, 359, Article 130427.
- Munjiza, A. A. (2004). *The combined finite-discrete element method, plus 0.5em minus 0.4em* John Wiley & Sons.
- Nemec, D., & Levec, J. (2005). Flow through packed bed reactors: 1. Single-phase flow. *Chemical Engineering Science*, 60(24), 6947–6957.
- Niegodajew, P., Durajski, A. P., Rajca, P., Gruszka, K. M., & Marek, M. (2022). Experimental and numerical study on the orientation distribution of cylindrical particles in random packed beds. *Chemical Engineering Journal*, 432, Article 134043 [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1385894721056175>.
- Noorman, S., van Sint Annaland, M., & Kuipers. (2007). Packed bed reactor technology for chemical-looping combustion. *Industrial & Engineering Chemistry Research*, 46(12), 4212–4220.
- Ohshima, N., Yanagi, K., & Miyoshi, H. (1997). Packed-bed type reactor to attain high density culture of hepatocytes for use as a bioartificial liver. *Artificial Organs*, 21(11), 1169–1176.
- Ong, C. J., & Gilbert, E. G. (2001). Fast versions of the gilbert-johnson-keerthi distance algorithm: Additional results and comparisons. *IEEE Transactions on Robotics and Automation*, 17(4), 531–539.
- Papkov, V., Shadymov, N., & Pashchenko, D. (2024). Gas flow through a packed bed with low tube-to-particle diameter ratio: Effect of pellet roughness. *Physics of Fluids*, 36(2).
- Partopour, B., & Dixon, A. G. (2017). An integrated workflow for resolved-particle packed bed models with complex particle shapes. *Powder Technology*, 322, 258–272.
- Pourciau, B. (2006). Newton's interpretation of Newton's second law. *Archive for History of Exact Sciences*, 60(2), 157–207.
- Sakai, M., Shigeto, Y., Basinskas, G., Hosokawa, A., & Fuji, M. (2015). Discrete element simulation for the evaluation of solid mixing in an industrial blender. *Chemical Engineering Journal*, 279, 821–839.
- Sánchez, A. J. P., Sánchez, E. P., & Silva, R. M. S. (2016). Design of a packed-bed absorption column considering four packing types and applying matlab. *Nexo Revista Científica*, 29(2), 83–104.
- P. Siemens, "Star-ccm+ homepage."
- Smilauer, V., Catalano, E., Chareyre, B., Dorofenko, S., Duriez, J., Gladky, A., Kozicki, J., Modenese, C., Scholtès, L., Sibille, L., et al. (2010). *Yade documentation*. the yade project.
- Thabet, A., & Straatman, A. G. (2018). The development and numerical modelling of a representative elemental volume for packed sand. *Chemical Engineering Science*, 187, 117–126.
- Theuerkauf, J., Witt, P., & Schwesig, D. (2006). Analysis of particle porosity distribution in fixed beds using the discrete element method. *Powder Technology*, 165(2), 92–99.
- Tong, K., Yang, L., Du, X., & Yang, Y. (2020). Review of modeling and simulation strategies for unstructured packing bed photoreactors with cfd method. *Renewable and Sustainable Energy Reviews*, 131, Article 109986.
- Wang, Y., Brannock, M., Cox, S., & Leslie, G. (2010). Cfd simulations of membrane filtration zone in a submerged hollow fibre membrane bioreactor using a porous media approach. *Journal of Membrane Science*, 363(1–2), 57–66.
- Xiuping, L., Youzhi, L., Zhiqiang, L., & Xiaoli, W. (2008). Continuous distillation experiment with rotating packed bed. *Chinese Journal of Chemical Engineering*, 16(4), 656–662.
- Yehia, H. M. (2022). *Rigid body dynamics. Advances in mechanics and mathematics*. Cham: Birkhäuser.
- Zhang, R., Wang, Y., Ren, H., Li, W., Tang, Z., & Zhang, D. (2024). Multi-scale study of turbulent mass transfer process in different shaped-adsorbent packed bed. *International Journal of Heat and Mass Transfer*, 218, Article 124810.
- Zhong, W., Yu, A., Liu, X., Tong, Z., & Zhang, H. (2016). Dem/cfd-dem modelling of non-spherical particulate systems: Theoretical developments and applications. *Powder Technology*, 302, 108–152.